Verification and Validation Report: Software Engineering

Team 13, Speech Buddies
Mazen Youssef
Rawan Mahdi
Luna Aljammal
Kelvin Yu

March 9, 2026

1 Revision History

Date	Version	Notes
March 8th, 2026	1.0	Added Non-Functional Performance tests
Date 2	1.1	Notes

2 Symbols, Abbreviations and Acronyms

symbol	description
T	Test

[symbols, abbreviations or acronyms – you can reference the SRS tables if needed —SS]

Contents

1 Revision History **i**

2 Symbols, Abbreviations and Acronyms **ii**

3 Functional Requirements Evaluation **1**

4 Nonfunctional Requirements Evaluation **1**

4.1 test-PF-1: ASR Latency Benchmark 1

4.1.1 test-PF-2: Command Pipeline Latency 3

4.1.2 test-PF-3: Accuracy & Precision (Stationary Noise) . . 5

4.2 Usability 6

4.3 Performance 6

4.4 etc. 6

5 Comparison to Existing Implementation **6**

6 Unit Testing **6**

7 Changes Due to Testing **7**

8 Automated Testing **7**

9 Trace to Requirements **8**

10 Trace to Modules **8**

11 Code Coverage Metrics **9**

List of Tables

1 ASR latency benchmark results for 20 audio samples 2

2 Command pipeline latency results for 20 audio samples 4

3 Traceability Between Unit Tests and MIS Modules 8

List of Figures

1 Terminal report of unit test result by module test. The bottom line summarizes the number of tests passed. 7
2 Terminal report of unit test coverage by module. Note that only modules deemed in scope, as mentioned in the VnV Plan are included 9

This document ...

3 Functional Requirements Evaluation

4 Nonfunctional Requirements Evaluation

4.1 test-PF-1: ASR Latency Benchmark

Type Automated, Dynamic

Test Case Derivation PRF-13.1

Test Summary The ASR latency benchmark was executed using 20 audio samples spoken by various speakers. For each sample, latency was measured from the marked end-of-speech point to the time at which the transcript became available. The measured latencies were recorded and averaged to evaluate performance against the requirement threshold.

Table 1: ASR latency benchmark results for 20 audio samples

#	**Sample Text**	**Length (words)**	**Latency (s)**
1	open google	2	5.81
2	google.com	1	6.30
3	search Wikipedia about paris	4	6.08
4	navigate to paris's sport section	5	5.93
5	open gmail	2	4.73
6	open google and search for keyboards	6	5.03
7	go on bestbuy and search for newest game	8	5.43
8	search software engineering jobs	4	4.48
9	search how to boil pasta	5	5.21
10	search used honda civic	4	6.08
11	search toronto raptors schedule	4	6.50
12	go to the history section of the canada page	9	7.01
13	search apple stock price	4	4.81
14	search directions to union station	5	5.21
15	open reddit	2	6.00
16	play jazz music on youtube	5	6.03
17	search restaurants near me	4	5.95
18	open wikipedia	2	4.83
19	search mcmaster university	3	6.80
20	open ebay	2	5.00
		Average Latency	**5.66**

Test Data and Results

Calculation of Average Latency The average latency is calculated as:

$$\text{Average Latency} = \frac{\sum_{i=1}^{20} L_i}{20} = 5.66 \text{ s}$$

where L_i is the latency measured for sample i.

Actual Result The measured average latency across the 20 audio samples was 5.66 seconds.

Conclusion This test **does not fully satisfy** PRF-13.1, since the measured average latency exceeded the required threshold of 5 seconds.

4.1.1 test-PF-2: Command Pipeline Latency

Type Automated, Dynamic

Test Case Derivation PRF-13.2

Test Summary The command pipeline latency benchmark was run using the same 20 audio samples from the ASR latency test. For each sample, latency was measured from the point at which the transcript became available to the point at which the requested command result was completed by the system. The measured latencies were recorded and averaged to assess compliance with the performance requirement.

Table 2: Command pipeline latency results for 20 audio samples

#	**Sample Text**	**Length (words)**	**Latency (s)**
1	open google	2	8.92
2	google.com	1	9.41
3	search Wikipedia about paris	4	12.87
4	navigate to paris's sport section	5	14.35
5	open gmail	2	9.76
6	open google and search for keyboards	6	13.42
7	go on bestbuy and search for newest game	8	15.91
8	search software engineering jobs	4	12.26
9	search how to boil pasta	5	11.83
10	search used honda civic	4	13.74
11	search toronto raptors schedule	4	12.95
12	go to the history section of the canada page	9	18.62
13	search apple stock price	4	10.54
14	search directions to union station	5	14.08
15	open reddit	2	9.88
16	play jazz music on youtube	5	13.67
17	search restaurants near me	4	12.43
18	open wikipedia	2	9.15
19	search mcmaster university	3	11.96
20	open ebay	2	8.64
		Average Latency	**12.52**

Test Data and Results

Calculation of Average Latency

$$\begin{aligned}\text{Average Latency} &= \frac{1}{20}(8.92 + 9.41 + 12.87 + 14.35 + 9.76 + 13.42 + 15.91 + 12.26 + 11.83 + 13.74 \\ &\quad + 12.95 + 18.62 + 10.54 + 14.08 + 9.88 + 13.67 + 12.43 + 9.15 + 11.96 + 8.64) \\ &= 12.52 \text{ s}\end{aligned}$$

Actual Result The measured average command pipeline latency across the 20 audio samples was 12.52 seconds.

Conclusion This test satisfied the performance requirement, since the measured average latency was below the 15 second threshold.

4.1.2 test-PF-3: Accuracy & Precision (Stationary Noise)

Type Offline Evaluation

Test Case Derivation PRF-13.1, PRF-13.2

Test Summary This test was planned to evaluate ASR and command classification performance under stationary background noise conditions, including AC and fan noise mixed at 20 dB, 10 dB, and 5 dB SNR. The intended evaluation metrics were word recognition accuracy and command precision under noisy conditions.

Result This test was not executed during the current development cycle.

Rationale Robustness to stationary background noise was identified as a stretch goal for the project. At this stage, development and validation efforts were prioritized toward core functionality under standard operating conditions. As a result, noise robustness testing was deferred and is outside the current project scope.

Conclusion No result is reported for this test. Evaluation of performance under stationary noise remains future work.

4.2 Usability

4.3 Performance

4.4 etc.

5 Comparison to Existing Implementation

This section will not be appropriate for every project.

6 Unit Testing

Unit testing for *VoiceBridge* focused on the core backend modules defined in the MIS and prioritised in the V&V Plan: the Speech-to-Text Engine (M4), Command Orchestrator (M5), Browser Orchestrator (M7), Session Manager (M8), Real-time Audio Capture (M15), and the Feedback Module (M20). For each of these modules we implemented automated tests using `pytest` that cover normal behaviour for all public access routines, representative edge cases, and key error paths (for example, invalid configuration, unavailable external services, malformed inputs, and I/O failures). Heavy external dependencies such as cloud LLM APIs, browser automation libraries, audio hardware, ASR model checkpoints, and the filesystem are replaced with test doubles where appropriate, allowing the unit tests to run deterministically and without network or device access. As a result, the unit test suite can be run quickly and repeatedly during development and in CI, and it provides direct evidence that the main control and data-flow responsibilities of each module behave as specified in the MIS.

The following figure captures the automatically generated result which summarize the results of the unit tests.

```
$ pytest
========================================================== test session starts ==========================================================
platform win32 -- Python 3.13.12, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\Rawan Alamily\Downloads\capstone\voicebridge
plugins: anyio-4.12.1, asyncio-1.3.0, cov-7.0.0, html-4.2.0, metadata-3.1.1
asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 274 items

test\test_app\test_command_orchestrator.py ..............................................................................     [ 28%]
test\test_app\test_stt.py ...................................................                                                [ 46%]
test\test_control\test_browser_controller.py ............                                                                   [ 51%]
test\test_control\test_session_manager.py ......................                                                            [ 59%]
test\test_data\test_feedback_module.py ......................................                                               [ 74%]
test\test_inputs\test_audio_capture.py ......................................................................               [100%]

========================================================== 274 passed in 1.44s ==========================================================
```

Figure 1: Terminal report of unit test result by module test. The bottom line summarizes the number of tests passed.

7 Changes Due to Testing

[This section should highlight how feedback from the users and from the supervisor (when one exists) shaped the final product. In particular the feedback from the Rev 0 demo to the supervisor (or to potential users) should be highlighted. —SS]

8 Automated Testing

Automated tests are executed locally via `pytest` and in continuous integration using GitHub Actions. The CI pipeline runs the full Python unit test suite on every push and pull request, including tests under `test/test_app`, `test/test_control`, and `test/test_inputs`, and fails the build if any test fails or if the test run crashes due to missing dependencies. Where necessary, platform-specific tests (for example, those that depend on Windows event loop policies) are guarded with `skip` conditions so that the suite remains portable across Linux-based CI runners and local Windows development machines. In addition to unit tests, the pipeline runs static analysis and style tools (such as `ruff` and `eslint`/`prettier` in the frontend) to catch common errors and enforce consistency before code is merged.

9 Trace to Requirements

10 Trace to Modules

Table 3 summarises how the implemented unit test suites map back to the MIS modules and, indirectly, to the functional requirements they support. This table is intentionally aligned with the Unit Test Description section of the V&V Plan; in this report we only record modules for which unit tests were actually implemented.

Table 3: Traceability Between Unit Tests and MIS Modules

MIS Module	**Primary Responsibilities**	**Unit Test File(s)**
M4 Speech-to-Text Engine	Audio format validation, ASR model integration, transcription pipeline, personalization	`test/test_app/test_stt.py`
M5 Command Orchestrator	LLM prompt construction, response parsing, clarification handling, safety guardrails	`test/test_app/test_command_orchestrator.py`
M7 Browser Orchestrator	Delegating textual tasks to browser automation agent	`test/test_control/test_browser_controller.py`
M8 Session Manager	Lifecycle of shared browser session (start/stop/get)	`test/test_control/test_session_manager.py`
M15 Audio Capture Module	Microphone capture, VAD-based speech detection, utterance segmentation, audio statistics	`test/test_inputs/test_audio_capture.py`
M20 Feedback Module	Persistent storage of structured command feedback records	`test/test_data/test_feedback_module.py`

Modules in the presentation layer and any auxiliary utilities that are not listed in Table 3 were not unit-tested during this iteration, either because they are thin UI shells around backend APIs or because they were judged lower priority for the current MVP. Their behaviour is instead validated

through end-to-end system tests, manual exploratory testing, and usability evaluations described earlier in this document.

11 Code Coverage Metrics

Code coverage for the Python backend was measured using `pytest-cov`, which instruments the code during unit test runs and produces line- and branch-coverage reports. The following figures capture the final state of coverage at the end of the project. The primary goal was to achieve high coverage for the safety- and control-critical modules (M4–M5 and M7–M8) and reasonable coverage for supporting components such as M15, rather than maximising overall project-wide coverage.

```
_____________________________________________ coverage: platform win32, python 3.13.12-final-0
===============…============

Name                                    Stmts   Miss  Cover
-------------------------------------------------------------
src\app\command_orchestrator.py           116     11    91%
src\app\speech_to_text_engine.py           95     16    83%
src\control\browser_orchestrator.py        16      0   100%
src\control\session_manager.py             16      0   100%
src\data\feedback_store.py                 24      0   100%
src\input\audio_capture.py                173      0   100%
-------------------------------------------------------------
TOTAL                                     440     27    94%
```

Figure 2: Terminal report of unit test coverage by module. Note that only modules deemed in scope, as mentioned in the VnV Plan are included

References

Appendix — Reflection

The information in this section will be used to evaluate the team members on the graduate attribute of Reflection.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

3. Which parts of this document stemmed from speaking to your client(s) or a proxy (e.g. your peers)? Which ones were not, and why?

4. In what ways was the Verification and Validation (VnV) Plan different from the activities that were actually conducted for VnV? If there were differences, what changes required the modification in the plan? Why did these changes occur? Would you be able to anticipate these changes in future projects? If there weren't any differences, how was your team able to clearly predict a feasible amount of effort and the right tasks needed to build the evidence that demonstrates the required quality? (It is expected that most teams will have had to deviate from their original VnV Plan.)